

Introduction

Most student and small-team projects start out being tracked in a spreadsheet or a chat thread, and that works fine for about two weeks. After that, nobody remembers who is supposed to be working on what, a bug report turns into a comment buried in a group chat, and "done" means something slightly different to every person on the team. Tools like Jira exist precisely because a shared, structured system is far more reliable than everyone keeping their own mental notes.

ScrumLab is my attempt to build a small version of that kind of system myself, rather than just using an existing one, so that I could actually design the database behind it instead of treating the data layer as something that already exists. The application follows a fairly standard Scrum/Jira-style workflow: projects contain issues (epics, stories, tasks and bugs), issues move through a small set of statuses on a board, and sprints group issues into time-boxed chunks of work. None of that is new. What this report is really about is what sits underneath the interface: a relational schema built for Oracle Database, and a set of PL/SQL functions, procedures, triggers and views that keep the important rules of the system enforced at the database level rather than only inside the PHP application code.

Objectives

- Design a normalized relational schema in Oracle that can represent projects, project membership, teams, sprints, and the full issue lifecycle, including the epic → story/task → subtask hierarchy.
- Move the business rules that matter most (status changes, parent/child status rollups, membership role validation, and change auditing) into the database itself using PL/SQL, instead of leaving them as logic that only exists in the PHP layer.
- Build a working full-stack application on top of that schema (Laravel + Blade/Livewire on the front end, Oracle on the back end) so the design is exercised by an actual user interface and not just a set of isolated SQL scripts.
- Write automated tests that check application behavior and, indirectly, that the database-level rules (triggers, procedures) hold under normal use.

System Overview

ScrumLab is organized around a small number of core concepts, most of which map directly onto a table in the database:

- Users: anyone with an account. A user can own projects, be assigned issues, report issues, comment, and belong to any number of teams or projects with a different role in each.
- Projects: a workspace with a name, a short key (used as a prefix for issue keys, similar to Jira's "PROJ-123" style), a description, an owner, and a status.
- Project members: the link between a user and a project, carrying a role (admin, project owner, scrum master, developer, or viewer).
- Teams: an optional sub-grouping inside a project, useful when a project is large enough that not everyone works on everything.
- Sprints: time-boxed windows (start date, end date, goal, status) that issues can be pulled into from the backlog.
- Issues: the central entity. Every issue has a type (epic, story, task, bug), a status, a priority, and can optionally have a parent issue, a sprint, a team, an assignee, and a reporter. Bug-type issues additionally carry a few bug-report-specific fields (steps to reproduce, expected/actual result, environment, severity).
- Boards: not a table of their own; a board is just the set of issues for a project or sprint, grouped into columns by their status field.
- Comments and activity logs: discussion threads on an issue, and a running history of what changed and when.
- Notifications: assignment, comment, status-change and sprint alerts delivered to a user.

Keys and Constraints

Every table uses a surrogate primary key, `id`, typed as `VARCHAR2(36)`, which holds a UUID rather than an Oracle sequence-generated number. This was a deliberate choice rather than a default: the Eloquent models use a small **HasUuid** trait that generates the `id` in PHP before insert, and the same format is mirrored on the database side by the `fn_new_uuid()` function, so that rows created directly by a trigger (for example, an `activity_logs` row written by an audit trigger) use identical-looking keys to rows created by the application. It also means primary keys are never guessable or sequential, which is a small but nice side benefit for a multi-tenant system like this.

A handful of composite unique constraints exist to stop duplicate relationships rather than to serve as keys: `project_members` enforces one row per (`project_id`, `user_id`) pair, `team_members` does the same for (`team_id`, `user_id`), and `teams` enforces a unique (`project_id`, `name`) so two teams inside the same project cannot share a name. `projects.key` and `issues.key` are both unique on their own, since those are the human-facing identifiers (a project key like `SCRUM`, an issue key like `SCRUM-42`).

Foreign keys deliberately mix cascade and set-null behaviour, depending on whether the child row is meaningful without its parent:

- Deleting a project cascades down through its members, teams, sprints and issues - none of those mean anything without the project.
- Deleting a sprint or a team only sets `issues.sprint_id` or `issues.team_id` to `NULL`; the issue survives and simply goes back to being unscheduled.
- `issues.assignee_id` is SET `NULL` if the assigned user is removed, but `issues.reporter_id` cascades - an issue cannot exist without whoever reported it.
- `projects.owner_id` currently cascades on user deletion, so deleting a user removes every project they own. Fine for a course project, but a production system would reassign ownership or soft-delete the user instead.

Table Summary

Table Name	Purpose	Key constraints
users	Registered accounts; owns projects, reports/is assigned issues, comments	PK id; UNIQUE email
projects	A workspace: name, key, description, status, owner	PK id; UNIQUE key; FK owner_id → users
project_members	Membership of a user in a project, with a role	PK id; UNIQUE(project_id,user_id); FKs → projects, users
teams	Optional sub-group inside a project	PK id; UNIQUE(project_id,name); FK → projects
team_members	Membership of a user in a team, with a role	PK id; UNIQUE(team_id,user_id); FKs → teams, users
sprints	Time-boxed iteration: name, goal, dates, status	PK id; FK project_id → projects
issues	Epic/Story/Task/Bug; the central work item	PK id; UNIQUE key; FKs → projects, teams, sprints, users (x2), self (parent_issue_id)
comments	Discussion thread on an issue	PK id; FKs → issues, users
activity_logs	Change history for a project/issue	PK id; FKs → projects, issues, users (all nullable)
notifications	Assignment/comment/status/sprint alerts	PK id; polymorphic notifiable_type/notifiable_id

PL/SQL Programming

This was the part of the project I spent the most time on. The rule I tried to follow throughout was: if a piece of logic has to be correct no matter what touches the data (the Laravel app, a raw SQL script run from SQL Developer, or a future integration nobody has written yet), then it belongs in the database, not in a PHP controller. Four kinds of PL/SQL objects came out of that: functions, procedures, triggers, and views.

Functions

Function	Purpose
count_open_issues(project_id)	Counts issues in a project whose status is not 'done'; powers the dashboard
fn_sprint_velocity(sprint_id)	Sums the story points of 'done' issues in a sprint
fn_issue_cycle_time(issue_id)	Whole days from creation to completion; NULL if the issue is not done yet
fn_project_progress_pct(project_id)	Percentage of a project's issues currently marked 'done'
fn_project_health(project_id)	A coarse label ('complete', 'on_track', 'at_risk', 'behind', 'empty') derived from progress and overdue sprint work
fn_new_uuid()	Generates a lower-case dashed UUID matching the app's VARCHAR2(36) key format, for rows created by triggers

Procedures

Procedure	Purpose
update_issue_status(issue_id, status)	Validates and applies a status change; this is the only path (app or SQL Developer) that should ever change an issue's status
rollup_parent_issue_status(issue_id, new_status)	Recomputes a parent epic/story's status from the current state of all of its children
proc_log_error(context, message)	Writes a structured error entry via an autonomous transaction, shared by the procedures above

Triggers

Trigger	Purpose
trg_project_members_role_chk	BEFORE INSERT/UPDATE on project_members; rejects anything outside the five valid role values
trg_issues_status_rollup	AFTER an issue's status changes; calls rollup_parent_issue_status so a parent's status never drifts from its children's
trg_issues_audit	AFTER INSERT/UPDATE/DELETE on issues; writes an activity_logs row in the very same transaction

Views

View	Purpose
v_issue_full	Joins an issue with its project, sprint, team, assignee and reporter, and adds the computed cycle-time in days from fn_issue_cycle_time, so a controller can query one view instead of five joins
v_project_stats	One live row per project, built directly on top of the metric functions from Section 6.1; the dashboard reads from this instead of recomputing the numbers by hand in PHP

Challenges and Learnings

A few things did not go smoothly on the first attempt, and I think they're worth mentioning honestly rather than glossing over.

Oracle + PHP on Windows: the OCI8 extension needs a matching Instant Client and several DLLs in the right PHP folder, and a stray older Oracle client on PATH breaks it with a misleading “specified procedure could not be found” error. I wrote a setup script (composer run setup-oracle) that downloads the client, copies the DLLs and enables the extension, so the problem only had to be solved once.

Laravel's Oracle-agnostic schema builder: there is no Schema::create() equivalent for “also create this trigger,” so the migrations for functions, procedures, triggers and views are raw DB::unprepared() calls. That keeps everything in one migration history but loses the nicer schema-builder syntax for those files.

The self-referencing issue hierarchy: parent_issue_id references issues.id with ON DELETE SET NULL rather than CASCADE, so deleting a subtask can never cascade upward and delete its parent story. Getting the rollout procedure to treat the row being updated correctly was the trickiest single piece of PL/SQL in the project.

Technology Stack

Layer	Technology	Notes
Backend	Laravel (PHP 8.2)	Routing, controllers, Eloquent models, authentication scaffolding
Frontend	Blade, Livewire, Vite	Server-rendered views; Livewire drives the reactive board and the notification bell
Database	Oracle Database 21c XE	Accessed through the yajra/laravel-oci8 driver
Database logic	PL/SQL	Functions, procedures, triggers and views living inside the schema itself

Conclusion

Building ScrumLab ended up being less about the Laravel application than I originally expected, and more about the schema and the PL/SQL sitting underneath it, which, given that this is a database course project, is probably the right outcome. What exists now is a schema that is constraint-driven and, as far as I can tell, sits cleanly in third normal form, a set of PL/SQL functions, procedures, triggers and views that enforce the rules that actually matter (valid statuses, valid roles, consistent parent/child rollups, and a complete audit trail), a working full application built on top of that schema, and a test suite that exercises both the app's behavior and, indirectly, the database rules underneath it. A few things were left out on purpose because of time rather than difficulty, and would be reasonable next steps: burndown and velocity charts built on **fn_sprint_velocity**, real-time board updates using the Reverb broadcasting setup that is already partially wired into the app, more fine-grained permission checks per role rather than the current coarse admin/owner/scrummaster/developer/viewer split, and one or two additional reporting views to support sprint retrospectives.